

BABU BANARASI DAS

UNIVERSITY



2026-2027

SCHOOL OF COMPUTER APPLICATIONS

No SQL Project On

Student Management System Using MongoDB

Submitted to:

Mr. Harendra Singh

Department: BCA (DS & AI)

Date: 25th September

Submitted by:

Aditya Kumar Gautam

Roll no: 36

University roll no: 1250258039

ACKNOWLEDGEMENT

I would like to sincerely thank my teacher, Mr. Harendra Singh, for giving me the chance to work on this NoSQL project, “Student Management System Using MongoDB”.

Working on this project helped me understand the core ideas behind NoSQL databases and showed me how MongoDB is used to create, read, update, sort and delete data. I also learned how comparison operators and the logical operators AND and OR are used to filter documents.

I am grateful to my teacher for the guidance and support given to me while completing this project.

INTRODUCTION

MongoDB is a widely used NoSQL database that stores data as flexible documents instead of the fixed rows and columns used by relational databases. Each document is similar to a JSON object, which makes it easy to store and change records.

In this project a Student Management System has been built in MongoDB. The database holds the records of 50 students, with each record containing a roll number, name, age, marks and city. Common operations such as inserting, displaying, updating, deleting, sorting, limiting, counting and searching were carried out on this data, and the comparison operators \$eq, \$gt, \$lt, \$gte and \$lte were demonstrated along with \$and and \$or.

OBJECTIVES

- To understand what NoSQL databases are and why they are used.
- To learn the basic features of MongoDB.
- To create a database and a collection.
- To insert 50 student records.
- To perform CRUD operations on the data.
- To use comparison operators in queries.
- To sort, limit and filter documents.
- To understand the \$and and \$or logical operators.

1. CREATE DATABASE

MongoDB Query:

```
use StudentDB
```

Explanation: Creates the StudentDB database, or switches to it if it already exists.

```
> _MONGOSH  
> use StudentDB  
< switched to db StudentDB
```

2. CREATE COLLECTION

MongoDB Query:

```
db.createCollection("students")
```

Explanation: Creates a collection named students inside the database.

```
> _MONGOSH  
> db.createCollection("students")  
< { ok: 1 }
```

3. INSERT 50 STUDENTS

MongoDB Query:

```
db.students.insertMany([
  {roll:1, name:"Vivaan", age:19, marks:64, city:"Prayagraj"},
  {roll:2, name:"Anaya", age:19, marks:73, city:"Dehradun"},
  {roll:3, name:"Reyansh", age:21, marks:59, city:"Dehradun"},
  {roll:4, name:"Myra", age:18, marks:69, city:"Jaipur"},
  {roll:5, name:"Krish", age:18, marks:83, city:"Prayagraj"},
  {roll:6, name:"Navya", age:21, marks:64, city:"Delhi"},
  {roll:7, name:"Ayaan", age:19, marks:96, city:"Prayagraj"},
  {roll:8, name:"Saarvi", age:19, marks:53, city:"Delhi"},
  {roll:9, name:"Dhruv", age:20, marks:90, city:"Jaipur"},
  {roll:10, name:"Kiara", age:19, marks:93, city:"Noida"},
  {roll:11, name:"Atharv", age:19, marks:95, city:"Dehradun"},
  {roll:12, name:"Avni", age:19, marks:90, city:"Noida"},
  {roll:13, name:"Shaurya", age:20, marks:89, city:"Delhi"},
  {roll:14, name:"Prisha", age:18, marks:71, city:"Prayagraj"},
  {roll:15, name:"Daksh", age:19, marks:73, city:"Varanasi"},
  {roll:16, name:"Anvi", age:21, marks:89, city:"Delhi"},
  {roll:17, name:"Neel", age:20, marks:90, city:"Varanasi"},
  {roll:18, name:"Ruhi", age:18, marks:86, city:"Prayagraj"},
  {roll:19, name:"Lakshya", age:21, marks:55, city:"Delhi"},
  {roll:20, name:"Tara", age:20, marks:64, city:"Jaipur"},
  {roll:21, name:"Pranav", age:19, marks:53, city:"Dehradun"},
  {roll:22, name:"Vanya", age:19, marks:76, city:"Delhi"},
  {roll:23, name:"Hrithik", age:19, marks:76, city:"Bareilly"},
  {roll:24, name:"Aarohi", age:20, marks:66, city:"Dehradun"},
  {roll:25, name:"Samar", age:19, marks:93, city:"Dehradun"},
  {roll:26, name:"Ananya", age:19, marks:59, city:"Varanasi"},
  {roll:27, name:"Veer", age:19, marks:70, city:"Dehradun"},
  {roll:28, name:"Mahi", age:19, marks:66, city:"Jaipur"},
  {roll:29, name:"Om", age:19, marks:61, city:"Jaipur"},
  {roll:30, name:"Zoya", age:20, marks:67, city:"Dehradun"},
  {roll:31, name:"Rohan", age:18, marks:95, city:"Bareilly"},
  {roll:32, name:"Jiya", age:19, marks:61, city:"Varanasi"},
  {roll:33, name:"Kunal", age:20, marks:91, city:"Varanasi"},
  {roll:34, name:"Sakshi", age:19, marks:85, city:"Varanasi"},
  {roll:35, name:"Naman", age:18, marks:83, city:"Bareilly"},
  {roll:36, name:"Trisha", age:19, marks:71, city:"Bareilly"},
  {roll:37, name:"Piyush", age:20, marks:75, city:"Varanasi"},
  {roll:38, name:"Divya", age:19, marks:79, city:"Dehradun"},
  {roll:39, name:"Chirag", age:18, marks:63, city:"Dehradun"},
  {roll:40, name:"Nisha", age:21, marks:62, city:"Jaipur"},
  {roll:41, name:"Yuvraj", age:19, marks:79, city:"Delhi"},
  {roll:42, name:"Bhumi", age:19, marks:93, city:"Noida"},
  {roll:43, name:"Tejas", age:19, marks:61, city:"Delhi"},
  {roll:44, name:"Radhika", age:20, marks:69, city:"Bareilly"},
  {roll:45, name:"Mayank", age:20, marks:83, city:"Delhi"},
  {roll:46, name:"Esha", age:21, marks:92, city:"Gorakhpur"},
  {roll:47, name:"Utkarsh", age:18, marks:69, city:"Prayagraj"},
  {roll:48, name:"Charu", age:20, marks:81, city:"Prayagraj"},
  {roll:49, name:"Vedant", age:20, marks:74, city:"Jaipur"},
  {roll:50, name:"Pari", age:18, marks:98, city:"Delhi"}
])
```

Explanation: Inserts all 50 student documents into the collection in a single command.

```
> _MONGOSH
```

```
> db.students.insertMany([
  {roll:1, name:"Vivaan", age:19, marks:64, city:"Prayagraj"},
```

```

    {roll:2, name:"Anaya", age:19, marks:73, city:"Dehradun"},
    {roll:3, name:"Reyansh", age:21, marks:59, city:"Dehradun"},
    {roll:4, name:"Myra", age:18, marks:69, city:"Jaipur"},
    {roll:5, name:"Krish", age:18, marks:83, city:"Prayagraj"},
    {roll:6, name:"Navya", age:21, marks:64, city:"Delhi"},
    {roll:7, name:"Ayaan", age:19, marks:96, city:"Prayagraj"},
    {roll:8, name:"Saanvi", age:19, marks:53, city:"Delhi"},
    {roll:9, name:"Dhruv", age:20, marks:90, city:"Jaipur"},
    {roll:10, name:"Kiara", age:19, marks:93, city:"Noida"},
    {roll:11, name:"Atharv", age:19, marks:95, city:"Dehradun"},
    {roll:12, name:"Avni", age:19, marks:90, city:"Noida"},
    ...
  < {
    acknowledged: true,
    insertedIds: { '0': ObjectId('6c1f8e52a4d7b0396e5c2a10'), ... '49':
ObjectId('6c1f8e52a4d7b0396e5c2a41') }
  }

```

4. DISPLAY ALL STUDENTS

MongoDB Query:

```
db.students.find()
```

Explanation: Retrieves the documents stored in the students collection.

```

> _MONGOSH

> db.students.find()

< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a10'),
  roll: 1,
  name: 'Vivaan',
  age: 19,
  marks: 64,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a11'),
  roll: 2,
  name: 'Anaya',
  age: 19,
  marks: 73,
  city: 'Dehradun'
}
... (48 more documents; output cropped)

```

5. EQUAL TO — \$eq

MongoDB Query:

```
db.students.find({marks: {$eq: 79}})
```

Explanation: Finds the students whose marks are exactly 79.

```
> _MONGOSH
> db.students.find({marks: {$eq: 79}})
< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a35'),
  roll: 38,
  name: 'Divya',
  age: 19,
  marks: 79,
  city: 'Dehradun'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a38'),
  roll: 41,
  name: 'Yuvraj',
  age: 19,
  marks: 79,
  city: 'Delhi'
}
```

6. GREATER THAN — \$gt

MongoDB Query:

```
db.students.find({marks: {$gt: 85}})
```

Explanation: Finds the students whose marks are greater than 85.

```
> _MONGOSH
> db.students.find({marks: {$gt: 85}})
< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a16'),
  roll: 7,
  name: 'Ayaan',
  age: 19,
  marks: 96,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a18'),
  roll: 9,
  name: 'Dhruv',
  age: 20,
  marks: 90,
  city: 'Jaipur'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a19'),
  roll: 10,
  name: 'Kiara',
  age: 19,
  marks: 93,
  city: 'Noida'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a1a'),
  roll: 11,
  name: 'Atharv',
  age: 19,
  marks: 95,
  city: 'Dehradun'
}
... (11 more documents; output cropped)
```

7. LESS THAN — \$lt

MongoDB Query:

```
db.students.find({marks: {$lt: 60}})
```

Explanation: Finds the students whose marks are less than 60.

```
> _MONGOSH
> db.students.find({marks: {$lt: 60}})
< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a12'),
  roll: 3,
  name: 'Reyansh',
  age: 21,
  marks: 59,
  city: 'Dehradun'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a17'),
  roll: 8,
  name: 'Saanvi',
  age: 19,
  marks: 53,
  city: 'Delhi'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a22'),
  roll: 19,
  name: 'Lakshya',
  age: 21,
  marks: 55,
  city: 'Delhi'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a24'),
  roll: 21,
  name: 'Pranav',
  age: 19,
  marks: 53,
  city: 'Dehradun'
}
... (1 more document; output cropped)
```

8. GREATER THAN OR EQUAL TO — \$gte

MongoDB Query:

```
db.students.find({marks: {$gte: 92}})
```

Explanation: Finds the students whose marks are 92 or above.

```
> _MONGOSH
> db.students.find({marks: {$gte: 92}})
< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a16'),
  roll: 7,
  name: 'Ayaan',
  age: 19,
  marks: 96,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a19'),
  roll: 10,
  name: 'Kiara',
  age: 19,
  marks: 93,
  city: 'Noida'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a1a'),
  roll: 11,
  name: 'Atharv',
  age: 19,
  marks: 95,
  city: 'Dehradun'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a28'),
  roll: 25,
  name: 'Samar',
  age: 19,
  marks: 93,
  city: 'Dehradun'
}
... (4 more documents; output cropped)
```

9. LESS THAN OR EQUAL TO — \$lte

MongoDB Query:

```
db.students.find({marks: {$lte: 65}})
```

Explanation: Finds the students whose marks are 65 or below.

```
> _MONGOSH
> db.students.find({marks: {$lte: 65}})
< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a10'),
  roll: 1,
  name: 'Vivaan',
  age: 19,
  marks: 64,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a12'),
  roll: 3,
  name: 'Reyansh',
  age: 21,
  marks: 59,
  city: 'Dehradun'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a15'),
  roll: 6,
  name: 'Navya',
  age: 21,
  marks: 64,
  city: 'Delhi'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a17'),
  roll: 8,
  name: 'Saanvi',
  age: 19,
  marks: 53,
  city: 'Delhi'
}
... (9 more documents; output cropped)
```

10. UPDATE OPERATION

MongoDB Query:

```
db.students.updateOne({roll:8}, {$set:{marks:96}})
```

Explanation: Updates the marks of the student having roll number 8 to 96.

```
> _MONGOSH

> db.students.updateOne(
  {roll:8},
  {$set:{marks:96}}
)

< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
> _MONGOSH

> db.students.find({roll:8})

< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a17'),
  roll: 8,
  name: 'Saanvi',
  age: 19,
  marks: 96,
  city: 'Delhi'
}
```

11. DELETE ONE OPERATION

MongoDB Query:

```
db.students.deleteOne({roll:15})
```

Explanation: Deletes one document matching roll number 15.

```
> _MONGOSH

> db.students.deleteOne({roll:15})

< {
  acknowledged: true,
  deletedCount: 1
}
```

```
> _MONGOSH

> db.students.find({roll:15})

<
```

12. DELETE MANY OPERATION

MongoDB Query:

```
db.students.deleteMany({marks: {$lt:58}})
```

Explanation: Deletes all the students having marks below 58.

```
> _MONGOSH
> db.students.deleteMany({marks:{$lt:58}})
< {
  acknowledged: true,
  deletedCount: 2
}
```

```
> _MONGOSH
> db.students.find({marks:{$lt:58}})
<
```

13. AND OPERATION — \$and

MongoDB Query:

```
db.students.find({$and:[{marks:{$gt:75}}, {age:20}]})
```

Explanation: Finds the students who satisfy both conditions: marks greater than 75 and age equal to 20.

```
> _MONGOSH

> db.students.find({
  $and:[
    {marks:{$gt:75}}, {age:20}
  ]
})

< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a18'),
  roll: 9,
  name: 'Dhruv',
  age: 20,
  marks: 90,
  city: 'Jaipur'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a1c'),
  roll: 13,
  name: 'Shaurya',
  age: 20,
  marks: 89,
  city: 'Delhi'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a20'),
  roll: 17,
  name: 'Neel',
  age: 20,
  marks: 90,
  city: 'Varanasi'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a30'),
  roll: 33,
  name: 'Kunal',
  age: 20,
  marks: 91,
  city: 'Varanasi'
}
... (2 more documents; output cropped)
```

14. OR OPERATION — \$or

MongoDB Query:

```
db.students.find({$or:[{city:"Varanasi"}, {city:"Prayagraj"}]})
```

Explanation: Finds the students who live in either Varanasi or Prayagraj.

```
> _MONGOSH

> db.students.find({
  $or:[
    {city:"Varanasi"}, {city:"Prayagraj"}
  ]
})

< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a10'),
  roll: 1,
  name: 'Vivaan',
  age: 19,
  marks: 64,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a14'),
  roll: 5,
  name: 'Krish',
  age: 18,
  marks: 83,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a16'),
  roll: 7,
  name: 'Ayaan',
  age: 19,
  marks: 96,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a1d'),
  roll: 14,
  name: 'Prisha',
  age: 18,
  marks: 71,
  city: 'Prayagraj'
}
... (9 more documents; output cropped)
```

15. SORT OPERATION

A. MongoDB Query:

```
db.students.find().sort({marks: 1})
```

Explanation: Sorts the students by marks from lowest to highest. 1 means ascending order.

```
> _MONGOSH
> db.students.find().sort({marks:1})
< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a12'),
  roll: 3,
  name: 'Reyansh',
  age: 21,
  marks: 59,
  city: 'Dehradun'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a29'),
  roll: 26,
  name: 'Ananya',
  age: 19,
  marks: 59,
  city: 'Varanasi'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a2c'),
  roll: 29,
  name: 'Om',
  age: 19,
  marks: 61,
  city: 'Jaipur'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a2f'),
  roll: 32,
  name: 'Jiya',
  age: 19,
  marks: 61,
  city: 'Varanasi'
}
... (43 more documents; output cropped)
```

B. MongoDB Query:

```
db.students.find().sort({marks: -1})
```

Explanation: Sorts the students by marks from highest to lowest. -1 means descending order.

```
> _MONGOSH

> db.students.find().sort({marks: -1})

< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a41'),
  roll: 50,
  name: 'Pari',
  age: 18,
  marks: 98,
  city: 'Delhi'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a16'),
  roll: 7,
  name: 'Ayaan',
  age: 19,
  marks: 96,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a17'),
  roll: 8,
  name: 'Saanvi',
  age: 19,
  marks: 96,
  city: 'Delhi'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a1a'),
  roll: 11,
  name: 'Atharv',
  age: 19,
  marks: 95,
  city: 'Dehradun'
}
... (43 more documents; output cropped)
```

16. LIMIT OPERATION

MongoDB Query:

```
db.students.find().limit(5)
```

Explanation: Displays only the first five documents of the collection.

```
> _MONGOSH
> db.students.find().limit(5)
< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a10'),
  roll: 1,
  name: 'Vivaan',
  age: 19,
  marks: 64,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a11'),
  roll: 2,
  name: 'Anaya',
  age: 19,
  marks: 73,
  city: 'Dehradun'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a12'),
  roll: 3,
  name: 'Reyansh',
  age: 21,
  marks: 59,
  city: 'Dehradun'
}
... (2 more documents; output cropped)
```

17. SEARCH BY CITY

MongoDB Query:

```
db.students.find({city:"Varanasi"})
```

Explanation: Finds all the students whose city is Varanasi.

```
> _MONGOSH
> db.students.find({city:"Varanasi"})
< {
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a20'),
  roll: 17,
  name: 'Neel',
  age: 20,
  marks: 90,
  city: 'Varanasi'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a29'),
  roll: 26,
  name: 'Ananya',
  age: 19,
  marks: 59,
  city: 'Varanasi'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a2f'),
  roll: 32,
  name: 'Jiya',
  age: 19,
  marks: 61,
  city: 'Varanasi'
}
{
  _id: ObjectId('6c1f8e52a4d7b0396e5c2a30'),
  roll: 33,
  name: 'Kunal',
  age: 20,
  marks: 91,
  city: 'Varanasi'
}
... (2 more documents; output cropped)
```

18. COUNT DOCUMENTS

MongoDB Query:

```
db.students.countDocuments()
```

Explanation: Counts the documents currently present in the collection (executed after the delete operations).

```
> _MONGOSH
> db.students.countDocuments()
< 47
```

MONGODB OPERATORS USED

Operator	Meaning	Example
\$eq	Equal to	marks: {\$eq:79}
\$gt	Greater than	marks: {\$gt:85}
\$lt	Less than	marks: {\$lt:60}
\$gte	Greater than or equal to	marks: {\$gte:92}
\$lte	Less than or equal to	marks: {\$lte:65}
\$and	All conditions true	\$and:[...]
\$or	At least one condition true	\$or:[...]

CONCLUSION

This project gave me practical experience with NoSQL databases using MongoDB. A Student Management System holding the details of 50 students was created, and a range of MongoDB operations were carried out on the data.

Through it I learned how to insert, retrieve, update and delete documents, how to sort data in ascending and descending order, and how the comparison operators \$eq, \$gt, \$lt, \$gte and \$lte work together with the logical operators \$and and \$or. Overall, the project showed me how MongoDB can be used to store and query student data efficiently.